

M13 / STUDY COMPANION

M13 Quick Revision Book

Standalone Zero-to-Job-Ready Data Engineer 2026

Study, practise, check and explain.

A complete learning route

Start with the lesson video. Read the worked explanation beside it, pause to reproduce the example, then solve the supplied task. The books contain your study notes; keep your own work and results as evidence.

Use the protected answers after saving your first attempt. A changed retry shows what you can now do independently. There is no forced timer during learning.

One course. M00-M19. 264 lessons and 50 project phases.

Contents

Tap a topic or use the PDF bookmarks. These numbers are the actual pages in this edition.

Read this first 5

M13 | DELTA LAKE + LAKEHOUSE 5

QUICK REVISION BOOK 5

DE13-01 | Data Lake vs Warehouse vs Lakehouse 5

Must be able to say 5

One failure to remember 5

One proof to save 5

DE13-02 | Why plain files are not tables 5

Must be able to say 5

One failure to remember 6

One proof to save 6

DE13-03 | Delta transaction log, ACID and table versions 6

Must be able to say 6

One failure to remember 6

One proof to save 6

DE13-04 | Create, read, update and delete 7

Must be able to say 7

One failure to remember 7

One proof to save 7

DE13-05 | Schema enforcement and schema evolution 7

Must be able to say 7

One failure to remember 8

One proof to save 8

DE13-06 | Time travel and version history 8

Must be able to say 8

One failure to remember 8

One proof to save 8

DE13-07 | MERGE, upserts and idempotent incremental loads 8

Must be able to say 8

One failure to remember 9

One proof to save 9

DE13-08 | Slowly Changing Dimension Type 2 with Delta 9

Must be able to say 9

One failure to remember 9

One proof to save 9

DE13-09 | Bronze, Silver and Gold lakehouse layers 10

Must be able to say 10

One failure to remember 10

One proof to save 10

DE13-10 | Compaction, OPTIMIZE, VACUUM and maintenance boundaries 10

Must be able to say 10

One failure to remember 10

One proof to save 11

DE13-11 | Integrated lakehouse pipeline and debugging checkpoint 11

Must be able to say 11

One failure to remember 11

One proof to save 11

Read this first

M13 | DELTA LAKE + LAKEHOUSE

QUICK REVISION BOOK

Fast retrieval after full study. This is not a substitute for the learner book or hands-on evidence.

Course runtime baseline: Spark 4.2.0 + Delta Lake 4.4.0. Live Delta/PySpark execution remains learner-PC proof unless explicitly recorded.

DE13-01 | Data Lake vs Warehouse vs Lakehouse

Lake, warehouse and lakehouse are architecture choices, not maturity ranks. A lakehouse keeps lake storage but adds table-level metadata and transaction behavior. Always name the required guarantee - current snapshot, update/delete, history, concurrency, schema control - before naming the technology.

Must be able to say

- Data lake: Object/file storage used for raw and processed data; flexibility is high, table guarantees depend on added systems.
- Data warehouse: Managed analytical database/platform with table semantics, SQL, governance and optimized serving.
- Lakehouse: Architecture that brings table/transaction capabilities to lake storage.
- Table format: Metadata + rules that define which data files belong to a table snapshot and how changes are committed.

One failure to remember

A common beginner failure is to call any collection of Parquet files a lakehouse. Parquet is a columnar file format; it does not by itself provide a transaction log or table history. Another failure is saying a lakehouse 'replaces warehouses.' Real architectures may use both, and the deciding factors include workload, governance, latency, team skills, interoperability and cost.

One proof to save

Save the three-design comparison, the changed compliance scenario, and a 60-90 second explain-back: 'Parquet files are not automatically a transactional table.' Mastery means you can state which layer supplies each guarantee without marketing language.

DE13-02 | Why plain files are not tables

A file is a storage object; a table is a logical contract. Delta's transaction log records committed actions and lets readers construct a snapshot. Current table membership is not determined by 'every Parquet file in the directory.'

Must be able to say

- Snapshot: The set of table metadata and data files that represent one committed version.

- Commit: An atomic recorded table change that creates a new version if it succeeds.
- AddFile / RemoveFile: Conceptual log actions that add a data file to or remove it from the current snapshot.
- `_delta_log`: Directory containing Delta transaction-log information for a table.

One failure to remember

Do not delete files manually because they 'look old.' Physical presence is not the same as logical membership. Do not use filesystem modified time as a business version. Do not write unrelated files inside a table directory and assume Delta will protect them from every maintenance operation. Table-managed directories need a clear ownership boundary.

One proof to save

Save the directory inspection, DESCRIBE HISTORY/history output, active-file paper exercise and changed retry. If Delta did not run on your PC, mark runtime PENDING and use the supplied simulator only as reasoning evidence.

DE13-03 | Delta transaction log, ACID and table versions

Delta versions represent committed table states. ACID is about transaction visibility and correctness, not immunity from failure. Debug from source -> job -> commit/history -> reconciled business output.

Must be able to say

- Atomicity: Expose all parts of a committed table change together, not a partial result.
- Consistency: Committed states respect table/protocol constraints and application invariants.
- Isolation: Concurrent operations see/produce controlled snapshots; conflicts are detected according to the protocol.
- Durability: Once committed, state survives normal process failure subject to underlying storage durability.

One failure to remember

Do not claim ACID means no job can fail. Jobs fail; ACID helps define what becomes committed and visible. Do not claim isolation means every writer is globally serialized from start to finish. Delta uses optimistic concurrency patterns. Do not equate schema enforcement with every business-quality rule - a string can be valid by schema and still contain an invalid customer code.

One proof to save

Save the version prediction, actual history, row/key totals, failure matrix and changed retry. Be able to explain why file count and version count are different concepts.

DE13-04 | Create, read, update and delete

Create/read/update/delete are table operations that produce committed snapshots. Precision comes from predicate preview, row/key/amount reconciliation and history evidence. Logical deletion and physical retention are different concerns.

Must be able to say

- DML: Data manipulation operations such as insert/update/delete/merge.
- Predicate: Boolean condition selecting rows to change.
- Logical delete: Row is removed from current table state; historical files may still exist until cleanup.
- Rewrite: Columnar files containing changed rows may be rewritten because Parquet files are immutable objects.

One failure to remember

Never demonstrate UPDATE/DELETE on valuable data without a safe sandbox, predicate preview and recovery plan. Do not tell a privacy stakeholder that DELETE immediately erases historical storage; VACUUM/retention and governance policies matter. Avoid using overwrite as a casual substitute for a targeted correction when it increases blast radius.

One proof to save

Save candidate-population previews, before/after controls, history rows and the changed legal-retention design. Explain how you would recover from an overly broad DML operation before retention cleanup.

DE13-05 | Schema enforcement and schema evolution

Enforcement makes drift visible; evolution makes approved change possible. Never confuse 'supported by the engine' with 'safe for the business contract.' Validate schema, keys, null behavior and consumers.

Must be able to say

- Schema enforcement: Reject/guard writes that do not match the target table structure according to Delta/Spark rules.
- Schema evolution: Intentional change to the target schema, such as adding columns through supported mechanisms.
- mergeSchema: Write option that can merge compatible incoming schema changes.
- overwriteSchema: Option used in overwrite scenarios when replacing a schema intentionally; high blast radius.

One failure to remember

Do not globally enable auto-merge merely to eliminate schema errors. That can turn a useful failure signal into silent drift. Do not use overwriteSchema as a first response to one bad batch. Do not treat column presence as proof of valid data: a newly added field may be 100% null or populated with malformed values.

One proof to save

Save the intentional failed write, before/after schemas, key reconciliation, evolution request and changed rename plan. Mastery means you can defend when to reject, evolve or quarantine.

DE13-06 | Time travel and version history

Time travel reads old snapshots; restore changes current state via a new commit. Historical availability is bounded by retention and physical files. Record versions as part of reproducible pipeline evidence.

Must be able to say

- Time travel: Read a previous table snapshot by version or timestamp.
- History: Metadata describing committed table operations/versions.
- Retention: Policy controlling how long historical log/data files remain available for reconstruction.
- VACUUM: Maintenance operation that removes unreferenced data files older than a retention threshold.

One failure to remember

Do not promise time travel forever. Do not run VACUUM with aggressively low retention just to save local disk and then expect old versions to work. Do not confuse reading an old version with restoring it: a historical read is non-destructive; RESTORE creates a new current commit and can affect downstream incremental consumers.

One proof to save

Save version-specific reconciliations, history output, incident note and retention retry. Be able to distinguish read-old, restore-old and vacuum-old without mixing them.

DE13-07 | MERGE, upserts and idempotent incremental loads

MERGE expresses conditional target changes; idempotency requires deterministic source selection, stable keys, precedence rules and safe progress commits. Reconcile business state after both first run and retry.

Must be able to say

- MERGE: Conditional table operation combining matched and not-matched actions.
- Upsert: Update an existing key or insert a new key.

- Idempotent: Repeating the same logical input produces the same final business state.
- Deduplication boundary: Rule that ensures one authoritative source candidate per target key for a merge.

One failure to remember

Do not feed duplicate source keys into MERGE and assume Delta will choose the latest row for you. Ambiguous multiple source matches can fail or create nondeterministic logic depending on the operation/design. Resolve duplicates explicitly before MERGE. Do not advance an upstream checkpoint before the target commit is durable and validated - M08's candidate-checkpoint pattern still applies.

One proof to save

Save source duplicate check, first-run controls, retry controls, stale-update proof and duplicate-key retry design. Be ready to explain why 'we use MERGE' is not enough to claim exactly-once business results.

DE13-08 | Slowly Changing Dimension Type 2 with Delta

SCD2 preserves descriptive history by expiring old versions and inserting new ones. Delta provides MERGE/transaction mechanics; correctness comes from temporal invariants, deterministic change detection and source-key discipline.

Must be able to say

- Dimension: Descriptive entity table such as customer, product, employee.
- SCD Type 2: History-preserving pattern that inserts a new version when tracked attributes change.
- Effective-from / effective-to: Validity interval boundaries for each historical version.
- Current flag: Convenience marker identifying the active version.

One failure to remember

Do not update the old row's attributes in place; that destroys history. Do not insert a new current version without expiring the old one; that produces two current rows. Do not compare every technical column when detecting change - audit timestamps can create a new version every run. Define the tracked business attributes deliberately.

One proof to save

Save before/after dimension rows, invariant checks, idempotent retry proof and multi-change reasoning. Explain why one current row per business key is necessary but not sufficient - interval overlap checks matter too.

DE13-09 | Bronze, Silver and Gold lakehouse layers

Bronze preserves source evidence, Silver creates validated/conformed truth, Gold serves business use. The value is clear contracts and replay/reconciliation, not the medal names themselves.

Must be able to say

- Bronze: Raw/landing table layer preserving source fidelity and ingestion metadata.
- Silver: Validated/conformed layer with explicit quality, typing, dedupe and business-cleaning rules.
- Gold: Serving layer optimized for business use cases such as aggregates, marts or dimensional outputs.
- Replayability: Ability to rebuild downstream state from preserved upstream evidence.

One failure to remember

Do not transform Bronze so heavily that original source evidence is lost. Do not let Silver silently drop rows without a reconciliation path. Do not build Gold directly from raw sources when Silver defines the canonical cleaning/conformance rules unless the exception is deliberate and documented. And do not assume Bronze/Silver/Gold means three copies of every dataset forever; retention and design depend on requirements.

One proof to save

Save layer schemas, Bronze/Silver/quarantine counts, Silver/Gold totals, design note and privacy retry. Be able to trace one Gold number back to source evidence.

DE13-10 | Compaction, OPTIMIZE, VACUUM and maintenance boundaries

OPTIMIZE changes layout; VACUUM removes eligible unreferenced physical files. Both require before/after evidence, but only VACUUM can destroy old-version readability. Never trade recoverability for a demo shortcut.

Must be able to say

- Small-file problem: Too many small data files create metadata/opening overhead and inefficient scans.
- Compaction / bin-packing: Rewrite layout into fewer/larger files while preserving table rows.
- OPTIMIZE: Delta command/API for file compaction; can return metrics.
- VACUUM: Remove unreferenced data files older than retention threshold.

One failure to remember

Do not equate OPTIMIZE with removing duplicates; it changes physical layout, not business grain. Do not use VACUUM as a performance fix for bad query plans. Do not disable retention checks or set zero-hour retention in a beginner lab. Do not run maintenance during an unresolved corruption incident when historical versions may be needed for comparison/recovery.

One proof to save

Save pre/post compaction controls, metrics/history, retention proposal and lag retry. State clearly whether live OPTIMIZE/VACUUM was executed or only reviewed on your machine.

DE13-11 | Integrated lakehouse pipeline and debugging checkpoint

A trustworthy lakehouse pipeline is a chain of deterministic state transitions with preserved source evidence, transactional table commits, reconciliation and recoverable history. Debug the first boundary where controls diverge.

Must be able to say

- Run contract: Documented inputs, batch identity, expected invariants, outputs and completion conditions for one pipeline execution.
- Reconciliation: Independent comparison proving populations/totals are preserved or intentionally changed.
- Recovery point: Known source/table version from which the pipeline can safely replay/rebuild.
- Blast radius: Scope of data/consumers potentially affected by an operation.

One failure to remember

Do not call the checkpoint complete because every script exited zero. Deliberately break one case: duplicate source key, invalid amount, stale update or schema drift. Save the first failure, diagnose it, repair one cause, rerun and prove reconciliation. Do not hide the error by dropDuplicates without a deterministic rule or by overwriting the whole table.

One proof to save

Save the integrated run report, live commands/results, one deliberate failure, recovery proof, idempotent retry and 90-second teach-back. If any runtime step was not executed on your learner PC, label it PENDING instead of converting static/simulator QA into mastery.